UNIVERSITÄT ZU LÜBECK
INSTITUTE FOR SOFTWARE ENGINEERING
AND PROGRAMMING LANGUAGES

isp

On the usage of Agentic AI to provide support for the AI Clinical Pathway

Brainstorming & Discussion

Karam Kharraz Martin Leucker

Content

- The Clinical AI Pathway
- Why do we want to use AI Agents for the Clinical AI Pathway
- What are AI Agents, and how to design them?
- Brainstorming of AI Agents usage within the Clinical AI Pathway
- Conclusion

The Clinical AI Pathway (CAP)

- The Clinical AI Pathway Guide serves as a structured framework to help medical professionals develop and implement AI solutions from early ideas into real-world clinical use.
- The framework has a staged phase structure from 1 to 6, focusing on Technology Readiness Levels (TRLs) scored 0–9.
- Stages are numbered sequentially, but the process is non-linear, requiring iterative loops to refine and align with R&D clinical needs.

Key idea

Action and tools to obtain project artifacts instead of blueprints.

Clinical AI Path Artefact examples for Phase 1-3

Phase	Some Example Artifacts
1. Idea	- Project plan detailing clinical needs and expected impact - Stakeholder analysis identifying participants and roles - Budget and resource plan for development phases
2. Concept	- Requirements for usability, specificity, and sensitivity - Regulatory requirements: GDPR, Risk assessment - Model Development Plan and metric objectives
3. Developement	- Data Artifacts: Dictionary, Statistical Overview - Model Artifacts: Software Architecture, Prototype - Evaluation Artifacts: value and performance

Using AI agents in CAP for what?

Use AI agent to Assist the stakeholders in managing project artifacts following the CAP methodology:

- Initialize a shared versioning repository with 6 folders per phase and an artefact for each action in each phase, with the right access rights for each stakeholder type.
- Detect dependencies between artifacts and construct dependency graph.
- Brainstorm and assist in filling the artifacts.
- Review and check the consistency of each artifact.
- Propagate changes to ensure global coherence between all artifacts.
- But what are AI Agents?

What are AI Agents

OpenAI: AI agents are systems that can independently accomplish tasks on your behalf, often by running multi-step workflows with a high degree of autonomy.

Anthropic: AI agents are systems where large language models (LLMs) dynamically direct their own processes and tool usage to accomplish tasks, maintaining control over how they achieve goals rather than following fixed, pre-programmed paths.

Google: AI agents are software systems that use AI to pursue goals and complete tasks on behalf of users. They show reasoning, planning, and memory and have a level of autonomy to make decisions, learn, and adapt. They can learn over time and facilitate transactions and business processes. Agents can work with other agents to coordinate and perform more complex workflows.

Slide 1: Architecture Paradigms

Monolithic Agent (1 Agent)

- Single LLM handles all planning, tool execution, and reasoning.
- **Best for**: Narrow-scope, straightforward tasks.
- **Limitation**: Prompt size blow-up, context degradation, hallucination at scale.

Multi-Agent (Orchestration)

- Specialized roles: Orchestrator, Worker, Reviewer.
- **Best for**: Enterprise workflows requiring reliability, verification, and distinct logical boundaries.
- **Advantage**:
  - Isolates errors.
  - Reduces cognitive load on models.
  - Enables cheaper/faster sub-task execution.

How to design Agents: First you need an agentic Ecosystem

Ecosystem	Orchestration Approach	Key Technologies
Anthropic	Architectural patterns: Orchestrator-Workers and Evaluator-Optimizer.	Express orchestration logic (loops, conditionals) directly in Python rather than natural language.
Microsoft Copilot	Connects broad ecosystems via Generative Orchestration.	Routes tasks dynamically between Connected Agents using the Model Context Protocol (MCP).
Google Vertex	Focuses on enterprise-grade scale and interoperability.	Vertex AI Agent Builder: Uses the Agent2Agent protocol to securely collaborate.

How to design One AI agent? Core Components

- **Core:** An LLM, giving the main capability of understanding, reasoning, and decision-making.
- **Tuning:**
  1. Persona: Establishes the agent's domain expertise and perspective,
  2. Tasks: Provides the denotational tasks and expected outcome, and
  3. guardrails: Enforces hard boundaries for security, compliance, and deterministic behavior.
- **Memory model:** Short-term, long-term memory such as project repository . . .
- **Tools:** external interfaces that allow the LLM to communicate with the real world, such as APIs for getting data or to use verification tools.

CAP Strategy: Event-Driven Agent Architecture

Suggest an architecture architecture that shifts the focus from static planning to continuous artifact updates and verifcation.

- **Event-Driven Orchestration:** Artifact changes trigger a central Orchestrator: detect location of change and delegate the necessary updates in the right order.
- **Phase-Specific Workers:** Delegated updates are routed to specialized agents.
- **Continuous Compliance:** Operating under strict personas guarantees traceability. Traceability ensures every code modification is verified instantly using tools such as risk matrices and test protocols.

Orchestrator Pattern: Code over Prompts

Concept: Use programmatic routing instead of LLM.

```
def route_change(event):
  # 1. LLM extracts a structured list of tasks
  tasks = CAP_Orchestrator.get_plan(event)
  # 2. Python logic generates the right update order
  for task in tasks:
    if task.phase == "Phase_2":
      compliance_agent.run(task.data)
    elif task.phase == "Phase_3":
      architect_agent.run(task.data)
    elif task.phase == "Phase_4":
      verification_agent.run(task.data)
    else:
      halt_system("Unknown Phase!")
```

CAP Orchestrator

```
model "claude-3-5-sonnet"
persona "Clinical AI Pathway (CAP) Routing Manager."
task "Analyze the input `event` and return a JSON array of specific
     tasks required to maintain system integrity."
guardrails
  - "Format: Output MUST be a strictly formatted JSON array."
  - "Constraint: Do NOT execute artifact changes. Only generate the
       task delegation plan."
memory
  short_term "The exact artifact modification event payload."
  long_term "Global CAP dependency graph."
tools [dependency_graph_query, json_schema_validator]
```

CAP Phase 3: Architect Agent

```
model "claude-3-5-sonnet"
persona "Senior Software Architect and MLOps Engineer."
task "receive orchestrator instructions to implement verified
    code or structural changes and update the artifacts in Phase
     3."
guardrails
  - "Traceability: Every architecture component MUST map back to a
       specific Phase 2 requirement."
   - "Integrity: If the requested change breaks existing module
       interfaces, trigger FLAG_FOR_REVIEW."
memory
   short_term "active IDE diagram drafts and specific
       orchestrator task instructions."
   long_term "Internal repository and the approved system
       architecture baseline."

tools [python_data_profiler, code_repository_committer]
```

Conclusion and Possible plan

1. The usage of agent requires a careful and progressive adaptation with alot of guardrails and formal verification tools.
2. We need to find a Mockup project (TP3 or synthetic).
3. Design manually the dependency graph.
4. Choose An agentic ecosystem and an artifect architecture using CAP.
5. Design all necessary workers types as stackeholders in CAP.
6. Benchmark and document the experience to use as guidelines for TP2.